
type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin Lanjut]]

[!cornell] RNN Fundamentals and Sequential Data Motivation

Questions/Cues

- Mengapa RNN diperlukan untuk data berurutan?
- Perbedaan mendasar Feedforward vs RNN
- Bagaimana parameter sharing bekerja di RNN?
- Contoh aplikasi RNN di kehidupan nyata
- Masalah utama RNN tradisional

Reference Points

- Lecture_01_DFS.pptx (Slides 5-11, 26-28)
- Advanced_Algorithms.pdf (Halaman 142-144)

Konsep Jaringan Saraf Berulang

Recurrent Neural Network (RNN) adalah jenis jaringan saraf tiruan dengan sambungan umpan balik (feedback) yang memungkinkan informasi bertahan antar langkah waktu. Berbeda dengan jaringan feedforward yang hanya memproses input secara independen, RNN memiliki memori internal yang merekam konteks sekuensial. Mekanisme ini meniru cara manusia memahami bahasa - kita menginterpretasikan setiap kata berdasarkan urutan kata sebelumnya. Analogi: Bayangkan membaca novel sambil mencatat karakter dan plot di buku catatan. RNN bertindak seperti buku catatan ini, di mana setiap halaman (hidden state) berisi ringkasan cerita hingga bab tersebut. Saat membaca bab baru (input baru), kita memperbarui catatan berdasarkan halaman sebelumnya.

Motivasi Data Sekuensial

RNN dirancang khusus untuk data dengan ketergantungan temporal atau urutan, dimana:

1. **Data deret waktu** (time-series): Harga saham, data cuaca, lalu lintas
2. **Data simbolik berurutan**: Histori penelusuran web, urutan klik pengguna
3. **Data bahasa alami**: Teks, ucapan, terjemahan mesin

Contoh konkret: Prediksi jumlah

penumpang maskenerapkan RNN dapat mengidentifikasi pola musiman (liburan) dan tren jangka panjang, sementara model feedforward hanya melihat snapshot waktu tertentu tanpa konteks historis.

Arsitektur Dasar RNN

RNN memiliki tiga komponen utama: 1. **Input (x_t)**: Vektor fitur pada langkah waktu t 2. **Hidden state (h_t)**: Memori jaringan yang menyimpan informasi dari langkah sebelumnya 3. **Output (y_t)**: Prediksi pada langkah t Persamaan fundamental: $h_t = f(Ux_t + Wh_{t-1} + b)$ Dimana U , W adalah matriks bobot dan b adalah bias. Parameter ini **dibagikan** di semua langkah waktu, memungkinkan jaringan mempelajari pola berulang secara efisien. Karakteristik ini berbeda dengan CNN yang menggunakan parameter berbeda di setiap lapisan.

Keunggulan dan Tantangan

Kelebihan RNN: - Mampu menangani input/output dengan panjang variabel - Menangkap ketergantungan jarak jauh (teoretis) - Efisien untuk data sekuensial melalui parameter sharing **Tantangan Utama**: - **Vanishing gradient**: Gradien menyusut secara eksponensial saat backpropagation melalui waktu, menyulitkan pembelajaran ketergantungan jangka panjang - **Kekakuan memori**: RNN tradisional kesulitan memilih informasi mana yang perlu disimpan/dibuang - Komputasi sekuensial menyulitkan paralelisasi

[!cornell] ##### Summary

RNN merupakan solusi fundamental untuk pemrosesan data sekuensial dengan memperkenalkan **memori internal** melalui mekanisme hidden state. Arsitekturnya mengatasi keterbatasan model feedforward dengan **parameter sharing** di seluruh langkah waktu dan kemampuan menangkap konteks temporal. Meski memiliki keunggulan dalam tugas seperti prediksi deret waktu dan pemrosesan bahasa, RNN tradisional menghadapi tantangan utama dalam **pembelajaran ketergantungan jangka panjang** akibat masalah vanishing gradient.

[!ad-libitum]- Additional Information

Analisis Matematis Parameter Sharing

Misalkan kita memproses sequence panjang T : - Jumlah parameter FFNN = $O(T \times d)$ untuk d fitur - Jumlah parameter RNN = $O(d^2 + dh + h)$ dengan h dimensi hidden state

Pada sequence panjang (T besar), RNN jauh lebih efisien. Bukti formal kompleksitas: $\text{Total Parameter} = (d \times h) + (h \times h) + h = h(d + h + 1)$ Dimana d = dimensi input, h = unit hidden state.

Vanishing Gradient: Bukti Formal

Pertimbangkan gradien $\partial L / \partial W$ untuk loss L pada waktu T : $\partial L / \partial W = \sum_{t=1}^T \partial L / \partial h_t \times \partial h_t / \partial W$. $\partial h_t / \partial W = \prod_{k=1}^t \text{diag}(f'(net_k)) \times h_{k-1}$. Jika $|f'(net_k)| < 1$ (misal untuk aktivasi sigmoid), perkalian berturut-turut menyebabkan gradien menyusut secara eksponensial seiring bertambahnya T .

Aplikasi Khusus RNN

1. Generative Modelling:

2. Komposisi musik dengan mempelajari pola notasi
3. Penulisan puisi otomatis dengan mempertahankan skema rima

4. Sensor Fusion:

5. Integrasi data accelerometer, gyroscope, dan GPS untuk prediksi aktivasi manusia

6. Financial Microscopy:

7. Pemodelan high-frequency trading dengan resolusi milidetik

Self-Exploration Projects

1. **Prediksi Kualitas Udara:** Gunakan dataset <https://airnow.gov> untuk memprediksi PM2.5 24 jam ke depan berdasarkan data historis. Bandingkan performa RNN vs ARIMA.
2. **Generasi Lirik Lagu:** Latih RNN pada korpus lirik musisi tertentu, perhatikan bagaimana model menangkap gaya linguistik dan tema khas.
3. **Analisis Rantai Pasokan:** Modelkan ketergantungan antara permintaan produk, cuaca, dan hari libur menggunakan data ritel.

Tools dan Resources

- **Pustaka:** TensorFlow Time Series, PyTorch Lightning, Sktime
- **Dataset:** UCI Time Series Repository, Kaggle Sequential Datasets
- **Visualisasi:** TensorBoard Embedding Projector, Plotly Dynamic Plots

Further Reading

- Bengio, Y., et al. (1994). Learning Long-Term Dependencies with Gradient Descent is Difficult
- LeCun, Y. (2022). A Path Towards Autonomous Machine Intelligence
- Interactive RNN Tutorial: <https://distill.pub/2020/understanding-rnns/>

type: Note cssclasses: - cornell-notes

Back to [[Pembelajaran Mesin]]

[!cornell] Forward Propagation Mechanics in RNNs

Questions/Cues

- Mengapa RNN memiliki loop dalam strukturnya?
- Bagaimana proses unrolling pada RNN?
- Apa fungsi parameter sharing dalam RNN?
- Langkah-langkah menghitung hidden state (h_t)
- Perbedaan perhitungan y_t dan h_t
- Contoh numerik forward propagation

Reference Points

- IF3270 Pembelajaran Mesin Slides (Halaman 11-23)
- Goodfellow et al. Deep Learning (Chapter 10)

Mekanisme Dasar Propagasi Maju RNN

Recurrent Neural Networks (RNN) memiliki mekanisme propagasi maju yang unik karena sifat rekursifnya. Berbeda dengan jaringan feedforward, RNN memproses data sekuensial dengan mempertahankan hidden state (h_t) yang membawa informasi dari langkah waktu sebelumnya. Proses ini dijelaskan dengan persamaan: $h_t = \tanh(Ux_t + Wh_{t-1} + b_{xh})$ dimana: - U : Matriks bobot input ke hidden layer - W : Matriks bobot hidden state sebelumnya - b_{xh} : Bias untuk hidden layer Sebagai analogi, bayangkan membaca sebuah novel: otak kita (h_t) tidak hanya memproses halaman saat ini (x_t), tetapi juga mengingat konteks dari halaman sebelumnya (h_{t-1}).

Konstruksi Dataset Sekuensial

Dataset untuk RNN dibangun dengan memperhatikan urutan temporal. Misal untuk data penumpang maskapai: 3 Feature Dataset: $X_1=t-2$, $X_2=t-1$, $X_3=t$, $Y=t+1$ 112, 118, 132 → 129 118, 132, 129 → 121 Representasi ini memungkinkan RNN mempelajari pola temporal. Pada contoh klasifikasi teks, setiap karakter di-encode sebagai vektor one-hot: $A = [1,0,0,0]$, $B = [0,1,0,0]$ Sequence "AB" → $[[1,0,0,0], [0,1,0,0]]$

Perhitungan Langkah-demi-Langkah

Contoh **numerik** **timestep** **t1:**

Input: $x_t = [1, 0, 0, 0]$ (karakter 'A') Bobot: $U = [[0.1, 0.15, 0.2, 0.3], \dots]$, $W = [[0.5, 0.5, 0.5], \dots]$ $h_t = \tanh(Ux_t + Wh_{t-1} + b_{xh}) = \tanh([0.1, 0.15, 0.2] + [0, 0, 0] + [0.1, 0.1, 0.1]) = \tanh([0.2, 0.25, 0.3]) = [0.197, 0.245, 0.291]$ (nilai aktual) $y_t = \text{softmax}(Vh_t + b_{hy}) = \text{softmax}([0.256, 0.242, 0.242, 0.173]) = [0.257, 0.253, 0.253, 0.236]$ Output terprediksi 'B' (indeks 1: 0.253) sementara target sebenarnya 'B' ($[0, 1, 0, 0]$).

Ketergantungan Hidden State

Nilai hidden state saat ini (h_t) bergantung secara rekursif pada: 1. Input saat ini (x_t) 2. Hidden state sebelumnya (h_{t-1}) 3. Bobot U dan W yang dishare antar timestep Pada timestep t2: $x_t = [0, 1, 0, 0]$ (karakter 'B') $h_{t-1} = [0.197, 0.245, 0.291]$ $h_t = \tanh(Ux_t + Wh_{t-1} + b_{xh}) = \tanh([0.15, 0.2, 0.3] + [0.467, 0.467, 0.467] + [0.1, 0.1, 0.1]) = [0.549, 0.583, 0.645]$ Perhatikan bagaimana informasi dari 'A' (t1) memengaruhi perhitungan 'B' (t2).

[!cornell] ##### Summary **Forward propagation di RNN** melibatkan perhitungan rekursif hidden state (h_t) yang mengkombinasikan input saat ini (x_t) dengan konteks sebelumnya (h_{t-1}). Parameter **U dan W dishare** di semua timestep, memungkinkan model memproses sekuens panjang. Output (y_t) dihasilkan melalui transformasi linear dan softmax. Contoh numerik menunjukkan bagaimana informasi mengalir melalui empat timestep dengan hidden state yang berkembang secara bertahap.

[!ad-libitum]- Additional Information

Analisis Matematis Propagasi Maju

Representasi formal RNN untuk sekuens panjang T : $h_t = \sigma(Ux_t + Wh_{t-1} + b_h)$ $y_t = \text{softmax}(Vh_t + b_y)$ Dengan: - σ : Fungsi aktivasi (biasanya tanh atau ReLU) - Dimensi: $U \in \mathbb{R}^{d \times d_x}$, $W \in \mathbb{R}^{d \times d}$, $V \in \mathbb{R}^{d_y \times d}$

Kompleksitas komputasi untuk sekuens panjang T adalah $O(Td^2)$ karena perkalian matriks $W \cdot h_{t-1}$.

Implementasi Numerik Optimal

Optimasi penting dalam implementasi RNN: 1. **Inisialisasi Bobot**: Bobot harus diinisialisasi dengan skala tepat (mis. Xavier initialization) untuk menghindari vanishing/exploding gradients 2. **Batching Sekuensial**: Mengelompokkan sekuens

dengan panjang sama untuk efisiensi GPU 3. **Masking Zero-Pad:** Mengabaikan padding dalam sekuens tidak rata

Contoh kode pseudo:

```
python def forward_rnn(x_seq, h0): h = h0 for t in range(len(x_seq)): h = tanh(np.dot(U, x_seq[t]) + np.dot(W, h) + b_h) y[t] = softmax(np.dot(V, h) + b_y) return y, h
```

Tantangan Praktis

1. **Vanishing Gradients:** Untuk sekuens panjang (>50 timestep), gradien bisa mengekspensial kecil
2. **Presisi Numerik:** Perhitungan softmax untuk kelas besar (mis. 10000 kata) memerlukan trik stabilitas numerik
3. **Parallelisasi:** Perhitungan RNN sulit diparalelkan karena ketergantungan serial antar timestep

Proyek Eksplorasi Mandiri

1. Implementasikan RNN dari scratch dengan Numpy untuk prediksi deret waktu:
2. Gunakan data saham harian (contoh: dataset AAPL 10 tahun)
3. Bandingkan hasil dengan ARIMA dan LSTM
4. Eksperimen dengan inialisasi bobot:
5. Uji inialisasi acak vs Xavier vs orthogonal
6. Ukur kecepatan konvergensi dan loss akhir
7. Visualisasi aliran hidden state:
8. Gunakan PCA untuk proyeksi 2D hidden state
9. Animasi evolusi hidden state selama training

Bacaan Lanjutan

- "On the Difficulty of Training Recurrent Neural Networks" (Pascanu et al., 2013)
- "An Empirical Exploration of Recurrent Network Architectures" (Jozefowicz et al., 2015)
- "Visualizing and Understanding Recurrent Networks" (Karpathy et al., 2015)
- Tutorial interaktif: <https://distill.pub/2019/memorization-in-rnns/>

Back to [[Pembelajaran Mesin Lanjut]]

[!cornell] RNN Architectural Variations and Implementation

Questions/Cues

- Apa perbedaan arsitektur n-to-n dan many-to-many?
- Bagaimana implementasi many-to-one di Keras?
- Mengapa diperlukan Bidirectional RNN?
- Bagaimana menghitung parameter di RNN multilayer?
- Apa aplikasi praktis arsitektur one-to-many?

Reference Points

- Lecture_01_DFS.pptx (Slides 27-39)
- Advanced_Algorithms.pdf (Pages 142-150)

Klasifikasi Arsitektur RNN

Recurrent Neural Networks memiliki tiga pola arsitektur utama berdasarkan hubungan input-output: **1. One-to-Many**: Satu input menghasilkan sekuens output. Contoh: Image Captioning dimana satu gambar (input tunggal) menghasilkan kalimat deskriptif (output sekuensial). Arsitektur ini menggunakan encoder CNN untuk ekstraksi fitur gambar dan decoder RNN untuk pembuatan teks. **2. Many-to-One**: Sekuens input menghasilkan satu output. Contoh: Analisis sentimen teks dimana sekuens kata-kata (input) diklasifikasikan menjadi sentimen positif/negatif (output tunggal). Implementasi di Keras menggunakan lapisan SimpleRNN dengan `return_sequences=False`. **3. Many-to-Many**: Terbagi menjadi dua subtype: - **Sinkron**: Setiap timestep input memiliki output terkait (e.g., POS Tagging) - **Asinkron**: Output mulai dihasilkan setelah seluruh input diproses (e.g., Machine Translation)

Implementasi Many-to-One di Keras

Contoh implementasi analisis sentimen:

```
python from keras.models import Sequential from keras.layers import SimpleRNN, Dense model = Sequential() model.add(SimpleRNN(64, input_shape=(100, 300))) # 64 neuron, input 100 timestep model.add(Dense(1, activation='sigmoid')) # Output klasifikasi biner
```

 Penjelasan parameter: - `input_shape=(100, 300)`: 100 timestep dengan vektor fitur 300 dimensi - Lapisan SimpleRNN secara otomatis mengembalikan hanya output terakhir

Arsitektur Many-to-Many

Dua pendekatan implementasi: **1. Sinkron (Sequence Tagging):** `python model.add(SimpleRNN(32, return_sequences=True))`
`model.add(TimeDistributed(Dense(10)))` # Output untuk setiap timestep
 Digunakan untuk tugas seperti penandaan POS dimana setiap kata mendapatkan label. **2. Asinkron (Encoder-Decoder):** - **Encoder:** Memproses seluruh input sequence menjadi context vector - **Decoder:** Menggunakan context vector untuk menghasilkan output sequence

Bidirectional RNN

Mengkombinasikan dua RNN: - **Forward Layer:** Memproses data secara kronologis - **Backward Layer:** Memproses data secara terbalik Keunggulan: - Mencapture dependensi maju-mundur (e.g., dalam NLP: konteks kata dipengaruhi kata sebelum & sesudah) - Implementasi di Keras: `python from keras.layers import Bidirectional model.add(Bidirectional(SimpleRNN(32)))`

[!cornell] ##### Summary **RNN** memiliki variasi arsitektur utama: **one-to-many** (generasi sekuens), **many-to-one** (klasifikasi sekuens), dan **many-to-many** (translasi/penandaan). Implementasi praktis menggunakan **return_sequences** untuk kontrol output, sementara **Bidirectional RNN** meningkatkan kinerja dengan konteks dua arah. Perhitungan parameter bergantung pada jumlah neuron dan dimensi input-output.

[!ad-libitum]- Additional Information

Perhitungan Parameter Lanjut

Rumus umum parameter RNN layer: `Parameter = (input_dim + hidden_units + 1) × hidden_units` Contoh RNN multilayer: `python model = Sequential() model.add(SimpleRNN(64, input_shape=(50,1), return_sequences=True)) model.add(SimpleRNN(32, return_sequences=True)) model.add(SimpleRNN(16))`
 Perhitungan: - Layer 1: $(1 + 64 + 1) \times 64 = 4224$ - Layer 2: $(64 + 32 + 1) \times 32 = 3104$ - Layer 3: $(32 + 16 + 1) \times 16 = 784$

Optimasi Arsitektur

Teknik advanced: - **Stacked RNN:** Multiple layer RNN untuk menangkap pola hierarkis - **Skip Connections:** Menghubungkan layer tidak berurutan untuk mitigasi vanishing gradient - **Dilated RNN:** Memperkenalkan celah temporal untuk perluasan reseptif

Kasus Edge

- **Long Sequences:** Problem vanishing gradient pada data >100 timestep
- **Sparse Data:** Input dengan banyak nilai nol menyebabkan pembelajaran tidak efisien
- **Non-uniform Sampling:** Interval waktu tidak konsisten pada data time-series

Proyek Eksplorasi

1. Bangun model pemetaan musik ke notasi (many-to-many)
2. Implementasi RNN untuk prediksi harga saham multi-variabel
3. Bandingkan kinerja SimpleRNN vs GRU pada dataset terjemahan kecil

Tools & Sumberdaya

- **Framework:** Keras/TensorFlow, PyTorch Lightning
- **Dataset:** CMU Articulatory Corpus, Penn Treebank
- **Visualisasi:** TensorBoard, Netron

Bacaan Lanjut

- "Deep Learning" (Goodfellow dkk.) Bab 10: Sequence Modeling
- Dokumentasi Resmi Keras: Bidirectional RNN
- Paper: "Empirical Evaluation of Gated RNNs" (Chung dkk. 2014)
- Tutorial: WildML - RNN untuk Machine Translation